IMAGE LICENSED BY INGRAM PUBLISHING

# Quantum Computing for Optimization With Ising Machine

OPTIMIZATION PROBLEMS, WHICH involve finding the best solution from a set of possible solutions, are ubiquitous in various fields, from finance to engineering. Traditional algorithms sometimes struggle with these problems, especially when the solution space is vast, or the landscape is filled with numerous local minima. Quantum-inspired computing, which emulates quantum mechanical principles on classical hardware, emerges as a promising paradigm to address these challenges. This paper delves into two notable approaches: coherent Ising machines (CIM) and graphics processing unit (GPU)-accelerated simulated annealing. In essence,

*Digital Object Identifier 10.1109/MNANO.2024.3378485*
*Date of current version: 16 May 2024*

YEN-JUI CHANG, CHIN-FU NIEN, KUEI-PO HUANG, YUN-TING ZHANG, CHIEN-HUNG CHO, AND CHING-RAY CHANG

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 20:41:05 UTC from IEEE Xplore. Restrictions apply.

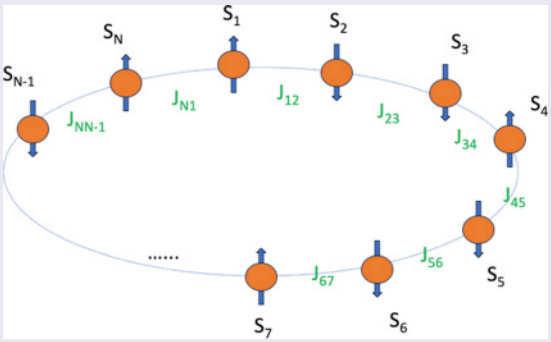


**FIGURE 1** A one-dimensional ising chain.

both methods offer innovative strategies to navigate the solution landscape, potentially bypassing the pitfalls of local optima and ensuring more efficient convergence to solutions. By harnessing the strengths of these quantum-inspired techniques, we can pave the way for enhanced computational capabilities in tackling complex optimization problems, even without a fault-tolerant quantum computer.

## INTRODUCTION

As Traditional computing continues to make remarkable progress, it is becoming evident that it is approaching the boundaries of its computational capabilities when confronted with intricate challenges. Complex problems such as optimization dilemmas or the simulation of quantum phenomena pose a formidable hurdle. In response to these challenges, the concept of quantum-inspired computing has emerged. While not genuinely quantum, it derives inspiration from the principles of quantum mechanics to unlock new dimensions of computational power. Leveraging a novel computing paradigm, quantum-inspired Ising machines offer a unique approach to solving integer programming problems. This is achieved by transforming these problems into equivalent representations within the Ising model framework. The Ising model [1], originally formulated to describe magnetic interactions within materials, represents atoms (or spins) in two states, typically denoted as +1 or -1. The interactions between these spins determine the overall energy of the system. Because many optimization problems can be mapped or translated into an energy minimization problem of the Ising model. In other words, the solution space of an optimization problem can be represented as spin configurations of the Ising model, and quantum or quantum-inspired methods can then be employed to find the configuration with the lowest energy, corresponding to the optimal solution of the original problem. By harnessing the inherent properties of the Ising model, these machines can efficiently navigate the solution space, potentially providing more effective and rapid solutions to complex integer programming challenges. Quantum-inspired algorithms leverage principles of quantum mechanics, like superposition and tunneling, to efficiently traverse the energy landscape of the Ising model. This allows these methods to potentially find optimal solutions more quickly and effectively, especially when the solution space is vast or peppered with numerous local optima. They offer a promising avenue for tackling complex optimization problems using classical computing resources, effectively bridging the gap between classical and quantum computing.

## COHERENT ISING MACHINE (CIM)

The Coherent Ising Machine (CIM) [2], [3], [4], [5] operates analogous to a one-dimensional Ising ferromagnetic chain found in condensed matter physics. This system comprises multiple interacting electron spins, each of which can adopt either an "up" or "down" state, showing in Figure 1.

Naturally, the system gravitates towards a low-energy state, characterized by its Hamiltonian: $H = -\sum_{i,j}^{N} J_{ij} S_i S_j - \sum_{i}^{N} h_i S_i$, $S_i$ denotes a spin, with possible values of ±1. Within the hardware of the CIM, a design utilizing optical fiber loops is employed. The process begins with a light source, specifically a laser beam known as the pump light, with a frequency of 2ω. This laser is directed onto a periodically-poled nonlinear optical crystal. Within the crystal, a nonlinear effect emerges, splitting the beam into two distinct light paths. These paths are referred to as the signal light and the idler light, both of which have a frequency of ω, half that of the pump light, and share the same polarization direction. The states of these two light beams exist in a "squeezed state" in quantum mechanics, serving as a qubit. This entire mechanism is recognized as a Degenerate Optical Parametric Oscillator (DOPO) process, showing in Figure 2. Building upon this foundation, as the power of the pump light is incrementally increased, the light produced by the DOPO remains in a "vacuum squeezed state" when the pump light's power is relatively low. However, once the pump light's power surpasses a certain threshold, the emitted light abruptly transitions to a "coherent state." This sudden shift from one state to another, termed a "quantum phase transition phenomenon," is analogous to the classical phase transition, like the transformation of water into ice. The DOPO-based CIM has demonstrated exceptional efficacy in addressing combinatorial optimization challenges [5], such as the Max-Cut problem, as evidenced by numerical simulations.

In the initial "vacuum squeezed state," the phase of the light remains undefined, but it becomes definable in the "coherent state." As illustrated in Figure 2, there is a "vacuum region" near the origin (0) below the oscillation threshold. Beyond this threshold, the light's phase bifurcates into two distinct states: phase 0 and π phase. Drawing upon the principles of the CIM, these phase states, 0 and π, can be mapped to the ±1 spins in the Ising model chain. This phase transition evolution of the entire system can then be harnessed to solve various optimization problems.


Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 20:41:05 UTC from IEEE Xplore. Restrictions apply.

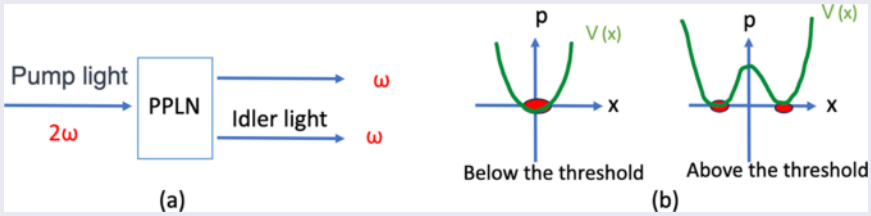


**FIGURE 2** Schematic diagram of the DOPO (a) the DOPO process (b) the distribution of DOPO.

Utilizing laser pulses, the CIM facilitates rapid computations by translating combinatorial optimization tasks into the Ising Model, an interacting spin model from statistical physics. This representation is then projected onto a quantum system, specifically an artificial spin network built using DOPO. By harnessing laser pulses within optical fibers as qubits, the CIM identifies the system's ground state, thereby solving the corresponding combinatorial nondeterministic polynomial (NP) optimization problems.

In real systems, coherent laser pulses emitted by the laser travel around in a fiber optic loop spanning thousands of kilometers. Each pulse, based on its phase, corresponds to a spin in the Ising model, representing a qubit. A portion of the light pulse circulating the fiber loop is extracted and its state is measured. Subsequently, the measurement results are fed into the high-speed electronic circuitry of an FPGA (Field-Programmable Gate Array), where they are programmatically assigned to rapidly compute the interactions within the Ising model. Following this, the computed results are swiftly relayed to a modulator, which adjusts the state of the feedback pulse. This feedback light pulse is then re-injected into the fiber optic loop, interacting with the original circulating pulse, facilitating the realization of spin interactions. This setup allows for the implementation of arbitrary spin couplings using a pair of measuring instruments and an FPGA, constructing a programmable Coherent Ising network. The ground state computation process of the spin chain is then realized by modulating the intensity of the pump light near its threshold. Based on the principle of minimal gain, when the gain in the Optical Parametric Oscillator (OPO) is less than the dissipation, the ground state energy of the Ising model is sought by increasing the pump power. As the gain parameter approaches the potential minimum loss, the entire system network undergoes an optical parametric oscillation phase transition. Thus, it oscillates in a precise or near-ground state. Further amplifying the pump light's gain to its threshold can continue to suppress erroneous oscillations in the excited state. Utilizing the aforementioned CIM hardware system, precise solutions to Ising problems can be determined. Recent publications [6] have assessed the performance of the CIM in solving problems with as many as 2,000 nodes. The scale of such a problem is immense, surpassing the processing capabilities of contemporary classical computers, making it nearly impossible to obtain precise solutions through brute-force computations.

QBoson Technology Co., Ltd. has pioneered the creation of the CIM, an innovative quantum computing apparatus specifically designed to address these complex issues. This device adeptly translates combinatorial optimization tasks into the interactive spin theoretical frameworks prevalent in statistical physics. QBoson, in partnership with entities including China Mobile Cloud, Huaxia Bank, and Qingda KeYue, has employed the CIM to validate solutions for real-world challenges. The system delivered results faster on average, outperforming classical algorithms and meeting some practical application standards. [7].

## SIMULATED ANNEALING

### *INTRODUCTION*

The simulated annealing (SA) algorithm is a heuristic optimization algorithm introduced by Kirkpatrick et al. [8] in the 1980s. The concept behind this algorithm is similar to approximate the cooling process of Boltzmann distribution. Initially, the system is heated to a high temperature, and then, after slowly decreasing the temperature, the system freezes to the low energy configuration, as shown in Figure 3. This cooling analogy is at the heart of the SA algorithm, as it allows the state to converge progressively toward optimal solutions for combinatorial problems as the temperature decreases appropriately [9], [10].

Generally, the SA algorithm comprises the Metropolis-Hasting (MH) algorithm with temperature scheduling. MH algorithm is a Markov chain Monte Carlo (MCMC) method [11] designed to generate samples from a probability distribution that is challenging to sample directly. In the MH algorithm, the requirements of the detailed balance and ergodicity result in the existence of a unique equilibrium distribution [10]. By exploiting the equilibrium feature of the MH algorithm, this allows us to approximate the target distribution and, hence, obtain the samples of the low energy states. If one considers the homogenous Markov chain with the Boltzmann distribution as the target distribution, when the temperature is very small, this can generate the optimal state of the problems. However, this often requires excessive time to reach equilibrium [10]. To circumvent this issue, as in the SA algorithm, one solution is to iterate the MH algorithm several times with a gradually decreasing temperature in each iteration.

Specifically, the procedure of the SA algorithm is shown in Figure 3. Suppose our optimization problem is to find the minimization of $f(x)$. First, given a fixed high-temperature value and initial state $x_0$, the state is updated according

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 20:41:05 UTC from IEEE Xplore. Restrictions apply.

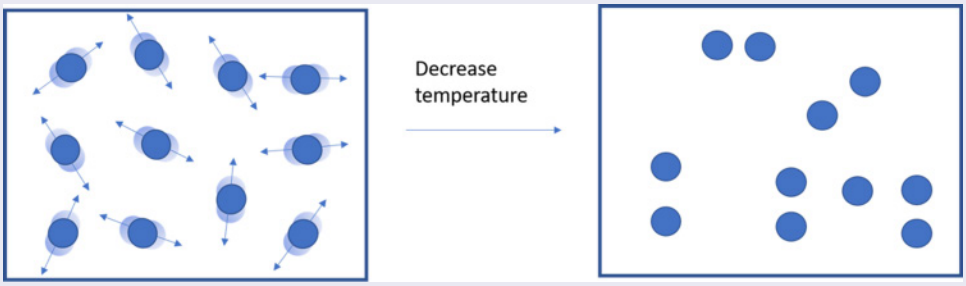


**FIGURE 3** When the temperature decreases gradually, the liquid system will cool down to a solid state.

> In contrast to CPU programming, GPU programmers are responsible for data allocation and movement. To achieve optimal performance, GPU programmers should be conscious of the underlying GPU architecture and how a GPU program interacts with it.

to the Metropolis criteria. Let $f(x_0)$ and $f(x_1)$ stand for the energy of neighboring states $x_0$ and $x_1$, if $f(x_1) < f(x_0)$, then the state is updated to $x_1$; conversely, if $f(x_1) > f(x_0)$, $f(x_1) > f(x_0)$ is accepted as the new state with the probabilities $\exp(\frac{f(x_1) - f(x_0)}{k_B T})$. After a sufficient number of updates, one can sample out the low energy state with some possibilities. When the temperature decreases properly, we can sample the lower energy state at a reasonable time cost [10]. In practice, one could leverage the Graphics Processing Unit (GPU) to accelerate the simulated annealing algorithm, which is introduced in the next section.

### *GRAPHICS PROCESSING UNIT (GPU) ACCELERATION FOR ISING COMPUTATIONS*

While some research teams are exploring the potential of harnessing quantum mechanics to construct quantum-based Ising machines, others are attempting to build quantum-inspired computers using classical computing technology. In recent times, an increasing number of established enterprises, startups, and research teams have demonstrated a growing interest in the development of these Ising solvers for addressing common real-world combinatorial optimization problems (COPs). Some businesses even develop specialized Ising solvers as commercial products. In contrast to fabricating hardware-based Ising machines, software solvers offer advantages such as lower development costs, high flexibility, and quicker time-to-market. As a result, the development of Ising solvers typically begins with software-based implementations.

GPU-based Ising solvers exhibit significantly higher performance compared to the Central Processing Unit (CPU). They also benefit from faster implementation times when compared to Field Programmable Gate Arrays (FPGA) and the production of physical hardware solvers. GPU technology's strength lies in its abundance of processing cores compared to the CPU, enabling GPUs to perform highly parallel computations.

While one may mistakenly think that working on GPU coding is simple, merely requiring a naive transformation of CPU code into a GPU-based version, several critical issues must be addressed to ensure both accuracy and efficiency. Programmers should pay close attention to how data is distributed among multiple cores and gathered to compute results; otherwise, it can lead to inefficient implementation or, worse, unexpected outcomes. To effectively harness GPU parallelism, programmers must have a solid grasp of the GPU parallel programming model when developing a GPU-based Ising solver. Below are several issues to consider for achieving a GPU-based Ising solver with high performance and solution quality:

### DATA MOVEMENT AND ALLOCATION

In contrast to CPU programming, GPU programmers are responsible for data allocation and movement. To achieve optimal performance, GPU programmers should be conscious of the underlying GPU architecture and how a GPU program interacts with it [12]. Figure 4 takes the programming model for Compute Unified Device Architecture (CUDA), one of the most widely used GPU programming models, as an example. A CUDA-capable GPU device comprises multiple streaming processors (SMs), each with multiple CUDA cores. When CUDA programmers


Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 20:41:05 UTC from IEEE Xplore. Restrictions apply.

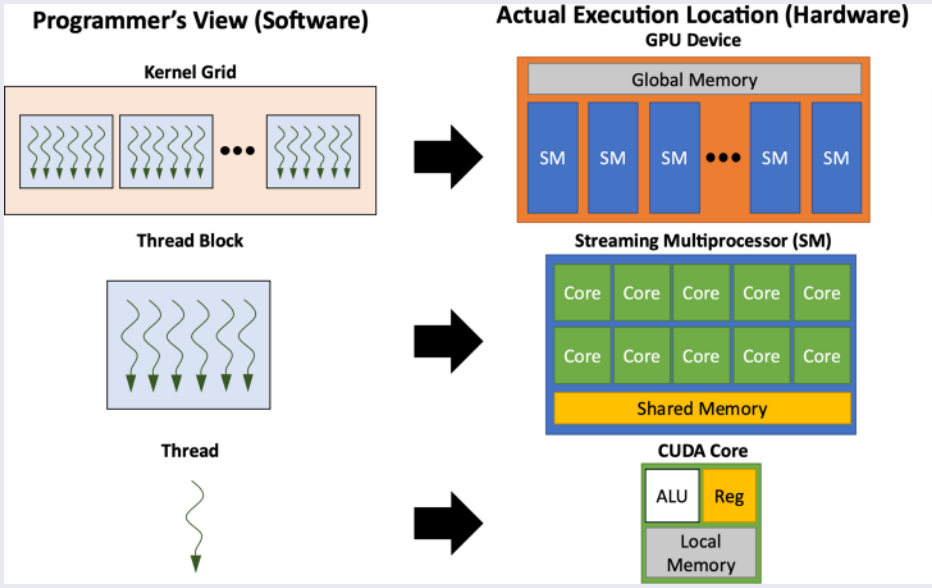


**FIGURE 4** CUDA programming model in the programmer's view versus the corresponding hardware architecture.

write code, they launch a CUDA "kernel grid" on a GPU device. A "grid" can be divided into multiple CUDA "thread blocks", which run at the SM level. Each "block" contains multiple CUDA "threads" to run on the CUDA cores. Unlike CPU programming, GPU programming requires the users to select parameters such as the number of blocks per grid or threads per block. If these numbers do not align with the underlying architecture, it leads to inefficient execution.

Another issue involves the memory units of CUDA-capable devices. Different memory units are found at various levels of the memory hierarchy. For example, "global memory" resides at the device level, accessible to all threads but with long latency. In contrast, "shared memory (SMEM)" resides at the SM level, where threads in the same SM can access data contents in SMEM relatively quickly. Accessing SMEM in a different SM involves data transfer between SMEMs and global memory, resulting in longer latency. To create an efficient GPU-based Ising solver, programmers need to carefully manage the partition of the coefficient matrix and the transfer of spin values to enhance GPU-based Ising machine performance.

To fully harness the benefits of latency hiding often requires programmers to optimize the kernel by adjusting the number of threads. This optimization allows the scheduler to execute latency hiding effectively and keep the available GPU cores busy.

## SCHEDULING OPTIMIZATIONS

Another critical concern involves scheduling optimization. Let's take the latency-hiding technique employed in a GPU-based annealer as an example [13]. Due to the diversity of memory units where data may reside, accessing data not cached in the memory units local to threads can be a time-consuming process, often taking hundreds of thousands of clock cycles. To minimize this latency in data movement, the hardware scheduler queues thread groups, referred to as "warps" in CUDA's programming model, which are still in the process of data transmission. This approach enables other "warps" that have completed data movement to be scheduled for computation, ensuring the efficient utilization of computational resources. To fully harness the benefits of latency hiding often requires programmers to optimize the kernel by adjusting the number of threads. This optimization allows the scheduler to execute latency hiding effectively and keep the available GPU cores busy.

## CO-OPTIMIZING SOLVER ALGORITHMS WITH GPU DEVICES

Several opportunities exist to enhance Ising algorithm performance on GPU devices. Co-optimizing solver algorithms

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 20:41:05 UTC from IEEE Xplore. Restrictions apply.

The solution of the Ising model can also be easily transformed into another NPcomplete (NPC) problem—the MAXCUT for solution. The MAX-CUT Problem is a key issue in the quantum world and is one of the main candidates to demonstrate the advantages of using quantum algorithms.

with GPUs can allow the Ising solver to exploit GPU characteristics better. For example, Han et al. [12] proposed accelerating an SA-based algorithm for IC floor planning on GPU devices. They address the challenge of parallelizing the SA algorithm on GPU devices, which is typically serial in nature, by breaking the dependency chain. This allows multiple annealing processes to run simultaneously on different threads within a GPU device.

## APPLICATION OF QUANTUM-INSPIRED COMPUTING: MAX-CUT PROBLEM

### *DESCRIPTION MAX-CUT PROBLEM*

The Maximum Cut Problem (Max-Cut Problem) is a combinatorial optimization problem commonly encountered in the fields of graph theory and computer science. In graph theory, a maximum cut refers to a cut whose size is equal to or larger than any other cut in the graph. In simpler terms, it's a way to partition the vertices of a graph into two distinct sets, $S$ and $T$, in a manner that maximizes the number of edges connecting $S$ to $T$. This problem is known as the "max-cut problem." To put it succinctly, the max-cut problem can be described as follows: the goal is to find a subset $S$ of the vertex set so that the number of edges between $S$ and its complementary subset is maximized. Alternatively, the objective is to identify a bipartite subgraph within the graph that contains the maximum possible number of edges.

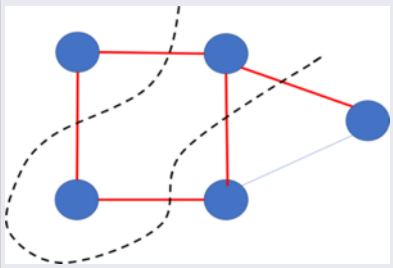

**FIGURE 5** A five-node example for the max-cut problem.

For a given optimization problem, we transform its objective function (or cost function) into a physical energy function (Hamiltonian) or the corresponding Ising model. We then search for its lowest energy state or ground state. The current approach involves converting the optimization problem into a QUBO (Quadratic Unconstrained Binary Optimization) problem. In general, the QUBO optimization problem is in the form

$$f(x) = x^T Qx$$

where $Q$ is a real value upper triangle matrix, and $x \in \{0,1\}^n$ ($n$ is the size of the problem.)

Likewise, we can express the Max-Cut problem into QUBO form. Given the $n$-nodes graph graph $G(V,E)$, where the sets $V$ and $E$ contain the whole vertices and edges in the graph and $|V| = n$. The variable $x_i \in \{0,1\}^n$ represents the status of the vertex. For a paired of connected vertices, $(x_i, x_j)$, when the cut crosses an edge between these two vertices, then $x_i$ and $x_j$ must have different values. As such, one can easily obtain the cost function for Max-Cut problem as

$$\sum_{i,j \in E} \left( x_i + x_j - 2x_i x_j \right).$$

By searching for the maximized value of this cost function, we can get the line which cut the maximized number of edges on the graph $G(V,E)$. For example, for the graph illustrated in Figure 5, the number of maximal cuts is 5.

### *RESULT*

The solution of the Ising model can also be easily transformed into another NP-complete (NPC) problem—the MAX-CUT for solution. The MAX-CUT Problem is a key issue in the quantum world and is one of the main candidates to demonstrate the advantages of using quantum algorithms. Utilizing Qboson's CIM alongside GPU-based SA solver, we derived the results for the max-cut problem.

QBoson Quantum Technology Co., Ltd. has pioneered the development of the CIM, a cutting-edge quantum computing device tailored to address such challenges. In an unprecedented advancement in the quantum computing arena, QBoson presents its CIM armed with 100 computational qubits. As shown in Figure 6(a), the visual depicts a complex network with a large number of interconnected nodes and edges, highlighted by red lines. This network likely represents the structure of the max-cut problem addressed by the Coherent Ising Machine. The nodes, marked by green and blue dots, might indicate different partitions or sets that result from the max-cut solution, which aims to maximize the sum of the weights of the edges that are 'cut' by the division. In Figure 6(b), this graph illustrates the performance of a Coherent Ising Machine in solving the max-cut problem. The x-axis measures the computation time in microseconds, and the y-axis indicates the number of cuts obtained. It is evident from the initial steep that the machine is capable of finding a solution very quickly, within the


Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 20:41:05 UTC from IEEE Xplore. Restrictions apply.

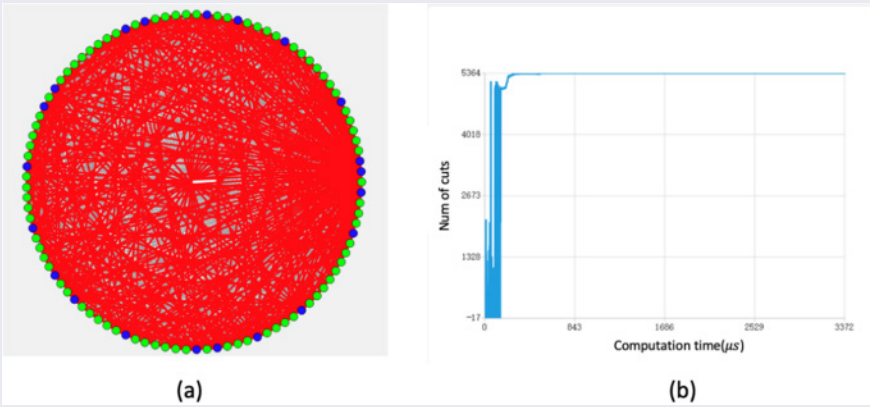


**FIGURE 6** The performance of the CIM on the MAX-CUT problem. (a) Find a division method that divides all vertices (vertex) into two groups(green and blue) while maximizing the number of cut edges (edge) (b) illustrates the computation times recorded when using the CIM to determine max-cut solutions, which notably occur within the span of hundreds of microseconds.

first few hundred microseconds. This pioneering machine is capable of achieving optimal solutions in a vast solution space equivalent to $2^{100}$, all within microseconds. In comparison, conventional computers would require tens of hours to solve similar scale problems. This breakthrough showcases an acceleration for practical application problems by the QBoson's CIM.

To show the effectiveness of another quantum-inspired computing technology, which is the GPU-based computation, we compare our GPU-based SA implementation with a CPU-based SA counterpart for the MAX-CUT problem using the G-Set dataset [14], as shown in Table 1. Our GPU implementation runs on an Nvidia 4090 GPU, while the CPU baseline runs on an Intel Core i9-12900KF CPU. We record the time for the two annealing solvers under the condition that both achieve the same accuracy level. The table shows that the GPU outperforms the CPU over 100 times in all scenarios. As the graph size increases, the speedup grows significantly, confirming John L. Gustafson's 1988 conclusion, known as Gustafson's Law [15], that the advantages of parallel computing increase with problem size. This is because the non-parallelizable part accounts for a smaller portion of larger-sized problems. For example, for a small graph, G1, the CPU requires about 1 minute to achieve a 98.247% accuracy.

**TABLE 1** Comparison of GPU-based and CPU-based ising solvers for the MAX-CUT problem on the G-SET dataset.

| G-SET DATASETS | GRAPH SIZE | GPU TIME (SEC) | CPU TIME (SEC) | SPEEDUP | ACCURACY (%) |
|---|---|---|---|---|---|
| G1 | 800 | 0.492 | 63 | 128.0 | 98.247 |
| G2 | 800 | 0.532 | 63 | 118.4 | 98.626 |
| G47 | 1000 | 0.667 | 125 | 187.4 | 97.252 |
| G36 | 2000 | 2.166 | 1096 | 506.1 | 96.745 |
| G55 | 5000 | 37.957 | 17386 | 458.2 | 94.821 |
| G63 | 7000 | 94.596 | 47423 | 501.4 | 97.100 |
| G70 | 10000 | 226.666 | 135827 | 599.4 | 94.195 |

In contrast, the GPU achieves the same accuracy level in just 0.492 seconds, resulting in a 128× speedup over the CPU-based baseline. This effect is more pronounced with a larger graph dataset, G70. Here, the CPU is required to run for one and a half days to achieve a solution quality similar to the GPU, which accomplishes it in 3.77 minutes. The CPU implementation is significantly less suitable in applications where timeliness is critical, such as high-frequency trading or real-time route planning, which demand rapid decision-making within a limited timeframe. In contrast, the GPU is more suitable due to its capacity for high-speed, parallel computation, making it the optimal choice for such scenarios. The speedup for such a large graph dataset is almost 600 times, significantly larger than the 128× speedup observed in the G1 dataset, verifying the conclusion drawn by Gustafson's Law. In summary, this experiment demonstrates the potential to harness GPU computing power to simulate Ising machines, thereby accelerating the solution process for COP problems, especially for large-size problems.

## CONCLUSION

Quantum-inspired computing, as exemplified by devices such as the Coherent Ising Machine (CIM) or GPU, presents a promising frontier for solving complex optimization challenges. While traditional computers often falter when faced with intricate nondeterministic polynomial-hard (NP-hard) problems like the MAX-CUT, quantum-inspired methodologies show potential for superiority. The adaptability of the Ising model, a foundational

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 20:41:05 UTC from IEEE Xplore. Restrictions apply.

construct in quantum mechanics, to the MAX-CUT problem underscores the dynamic potential of quantum approaches.

At the vanguard of this wave is QBoson Quantum Technology Co., Ltd., which has crafted the CIM boasting an impressive 100 computational qubits. This technological marvel navigates a vast solution space equivalent to $2^{100}$ in a fraction of the time that conventional computers might spend on analogous tasks. The remarkable speed advantage, exceeding a thousand times for certain applications, highlights the prowess of QBoson's CIM.

Additionally, GPU-based computing presents a compelling alternative. We compare the Nvidia RTX 4090 GPU implementation of the simulated annealing (SA) with that of the Intel Core i9-12900KF CPU on the G-Set dataset to find out that the GPU consistently outperforms the CPU in all tests, with performance improvements of over 128 times for smaller data sets compared to CPUs. In scenarios with larger data sizes, this advantage can surge to 600 times, making GPUs an adaptable choice for applications requiring rapid computations, such as high-frequency trading and real-time route determination. In essence, the power and efficiency of GPUs, especially when simulating Ising machines, hold significant promise for hastening solutions to combinatorial optimization conundrums, especially those of larger magnitudes.

Quantum-inspired computing can be a promising technique for combinatorial optimization problems before the advent of fault-tolerant quantum devices. Due to the limited coherent time and error-non-resilience qubits, current quantum devices cannot calculate large-scale problems. As such, emulating the quantum property on classical hardware could provide a possibility to show how quantum properties can benefit combinatorial optimization problems in the near term. Recently, the related fields have thrived and drawn much attention. Our article provides a clear and understandable introduction to several quantum-inspired algorithms, and we hope this can benefit those interested in this area.

## ACKNOWLEDGMENT

This work was supported by the National Science and Technology Council, Taiwan under NSTC Grant 112-2119-M-033-001, Grant 112-2221-E-182-031, and Grant 111-2221-E-182-025.

We thank QBoson for their insightful discussions and valuable contributions to this work. We want to thank the NVIDIA AI Technology Center for their insightful discussions and valuable contributions, which helped enrich this research.

## ABOUT THE AUTHORS

***Yen-Jui Chang*** (corresponding author: (d09222012@ntu.edu.tw)) is with the Department of Physics, National Taiwan University, Taipei 10617, Taiwan.

***Chin-Fu Nien*** (watchmannien@mail.cgu.edu.tw) is with the Department of Computer Science and Information Engineering, Chang Gung University, Taoyuan City 33302, Taiwan.

***Kuei-Po Huang*** (m1129031@cgu.edu.tw) is with the Department of Computer Science and Information Engineering, Chang Gung University, Taoyuan City 33302, Taiwan.

***Yun-Ting Zhang*** (b0929007@cgu.edu.tw) is with the Department of Computer Science and Information Engineering, Chang Gung University, Taoyuan City 33302, Taiwan.

***Chien-Hung Cho*** (f06222035@ntu.edu.tw) is with the Department of Physics, National Taiwan University, Taipei 10617, Taiwan.

***Ching-Ray Chang*** (crchang@phys.ntu.edu.tw) is with the Department of Physics, National Taiwan University, Taipei 10617, Taiwan, and also with Quantum Information Center, Chung Yuan Christian University, Taoyuan City 32023, Taiwan.

## REFERENCES

[1] S. G. Brush, "History of the Lenz-Ising model," *Rev. Modern Phys.*, vol. 39, pp. 883–893, 1967.

[2] P. L. McMahon et al., "A fully programmable 100-spin coherent ising machine with all-to-all connections," *Science*, vol. 354, no. 6312, pp. 614–617, 2016.

[3] T. Inagaki et al., "A coherent Ising machine for 2000-node optimization problems," *Science*, vol. 354, no. 6312, pp. 603–606, 2016.

[4] D. Pierangeli, G. Marcucci, and C. Conti, "Large-scale photonic ising machine by spatial light modulation," *Phys. Rev. Lett.*, vol. 122, no. 21, 2019, Art. no. 213902.

[5] Z. Wang et al., "Coherent Ising machine based on degenerate optical parametric oscillators," *Phys. Rev. A*, vol. 88, 2013, Art. no. 063853.

[6] T. Inagaki et al., "A coherent Ising machine for 2000-node optimization problems," *Science*, vol. 354, pp. 603–606, 2016.

[7] J. Wen et al., "Optical experimental solution for the multiway number partitioning problem and its application to computing power scheduling," *Sci. China Phys., Mechanics Astron.*, vol. 66, 2023, Art. no. 290313.

[8] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, pp. 671–680, 1983.

[9] E. Aarts and J. Korst, "Simulated annealing and boltzmann machines: A stochastic approach to combinatorial optimization and neural computing," Chichester, U.K.: Wiley, 1989.

[10] D. Bertsimas and J. Tsitsiklis, "Simulated annealing," *Stat. Sci.*, vol. 8, pp. 10–15, 1993.

[11] C. P. Robert, G. Casella, and G. Casella, *Monte Carlo Statistical Methods*, vol. 2. Berlin, Germany: Springer, 1999.

[12] Y. Han, S. Roy, and K. Chakraborty, "Optimizing simulated annealing on GPU: A case study with IC floorplanning," in *Proc. 12th Int. Symp. Qual. Electron. Des.*, 2011, pp. 1–7.

[13] L. Y. Barash, M. Weigel, M. Borovský, W. Janke, and L. N. Shchur, "GPU accelerated population annealing algorithm," *Comput. Phys. Commun.*, vol. 220, pp. 341–350, 2017.

[14] Y. Ye, "Gset." 2003. [Online]. Available: https://web.stanford.edu/~yyye/yyye/Gset/

[15] J. L. Gustafson, "Reevaluating Amdahl's law," *Commun. ACM*, vol. 31, no. 5, pp. 532–533, 1988.

N



Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 20:41:05 UTC from IEEE Xplore. Restrictions apply.